

31 — Webhooks and Event Notification

Requirement: R25 (Admin UI configuration), ETL pipeline completion notification **Phase:** K

What it is

When a long-running process finishes — like an ETL pipeline that takes minutes or hours to process thousands of records — other systems often need to know about it. There are two fundamental approaches to this problem: **polling** and **webhooks**.

Polling (the "Are we there yet?" pattern)

With polling, the client repeatedly asks the server "Are you done yet?" at regular intervals:

```
Client: Is the ETL done?      Server: No.  
(wait 30 seconds)  
Client: Is the ETL done?      Server: No.  
(wait 30 seconds)  
Client: Is the ETL done?      Server: Yes! Here are the results.
```

Polling is simple to implement but wasteful. Most of those requests return "no" — they consume bandwidth, server resources, and database connections for nothing. If you poll too frequently, you waste resources. If you poll too infrequently, you miss the completion by minutes.

Webhooks (the "Don't call us, we'll call you" pattern)

With webhooks, the relationship is inverted. The client says "Here is my URL. When you are done, send an HTTP POST to it." The server stores the URL and, when the event occurs, sends a single notification:

```
Client: Notify me at https://my-system.com/etl-done  
(hours pass, ETL runs)  
Server: POST https://my-system.com/etl-done {status: "complete", ...}
```

One request, exactly when it matters. No wasted calls, no delays.

Webhook anatomy

A webhook is just a regular HTTP POST request that the server sends to a URL that the client specified in advance. The payload is typically JSON containing information about the event. The key difference from a normal API call: the server is making the request, not the client. The server becomes a client of whoever registered the webhook URL.

Configurable webhook URL

In this project, the webhook URL is not hardcoded — it is stored in the `app_config` database table under the key `etl_webhook`. An admin can set or change it through the Admin UI's configuration panel. This means different deployments can notify different systems: a Slack channel webhook, a monitoring service, a custom dashboard, or nothing at all (if no URL is configured, the webhook simply does not fire).

Retry on failure

Network requests fail. The target server might be temporarily down, there might be a DNS resolution issue, or the request might time out. Robust webhook implementations retry failed deliveries. In this project, the `requests.post` call uses a 10-second timeout, and failures are logged as warnings rather than crashing the pipeline — the ETL completing successfully is more important than the notification reaching its destination.

Real-life analogy

Imagine you order a custom piece of furniture from a woodworking shop.

Polling is like driving to the shop every day and asking "Is my table ready?" Most days the answer is no, and you have wasted 30 minutes of driving. On the day it is ready, you might still arrive 12 hours after completion because you only check once a day.

A webhook is like giving the shop your phone number and saying "Text me when it's done." You go about your life, and the moment the table is finished, you get a text. One message, zero wasted trips, no delay.

The configurable URL is like being able to say "Actually, text my partner instead, they'll pick it up" — you can change who gets notified without the shop changing their process.

Retry on failure is like the shop trying to text you, getting no response (your phone was off), and trying again in a few minutes. If it still fails, they make a note in their log ("attempted to notify customer, phone unreachable") and move on — they do not throw away the table just because the notification failed.

Why we used it here

The ETL pipeline in this project processes hundreds of categories and companies through translation, AI enrichment, embedding, and database upsert. A full run can take significant time. External systems (monitoring dashboards, Slack channels, CI/CD pipelines) need to know when it finishes and whether it succeeded.

The design choices:

1. **Webhook over polling** — the ETL is a batch process that runs occasionally (not a continuously streaming service). Polling would waste resources during the long idle periods between runs. A webhook delivers the notification exactly once, at exactly the right time.
 2. **Configurable URL via `app_config`** — different environments need different notification targets. A development machine might point to a local endpoint. A staging environment might notify a Slack channel. Production might notify a monitoring service. Storing the URL in `app_config` (editable through the Admin UI) means no code changes or redeployment needed to change the notification target.
 3. **Fire-and-forget with logging** — the webhook is a best-effort notification, not a critical data path. If it fails, the ETL results are still safely in the database. The failure is logged so operators can investigate, but the pipeline does not retry indefinitely or roll back its work.
 4. **Summary payload** — the webhook sends the same completion summary that gets stored in `app_config` as `etl_progress`. This includes counts of processed, skipped, and failed records for both categories and companies, plus a timestamp. The recipient has enough information to decide whether action is needed.
-

Code walkthrough

1. Sending the webhook on ETL completion (`bilingual_etl/scripts/main_etl.py`)

The webhook is sent at the very end of the ETL pipeline, after all categories and companies have been processed:

```
def _send_etl_webhook(conn, summary: dict) -> None:
    try:
        with conn.cursor() as cur:
            # Look up the webhook URL from the app_config table.
            # The key "etl_webhook" stores a JSON object with a "url" field.
            # If no row exists or the URL is empty, skip silently.
            cur.execute(
                "SELECT value->>'url' FROM app_config WHERE key = 'etl_webhook'"
            )
            row = cur.fetchone()
            if not row or not row[0]:
                return # No webhook configured – nothing to do

            url = row[0]
            # Send the summary as a JSON POST request.
            # timeout=10 prevents hanging if the target is unresponsive.
            # The requests library handles JSON serialization automatically.
            http_requests.post(url, json=summary, timeout=10)
            logger.info(f"ETL webhook sent to {url}")
    except Exception as e:
        # Log the failure but do NOT re-raise.
        # The ETL completed successfully – the notification failing
        # should not be treated as a pipeline failure.
        logger.warning(f"ETL webhook failed: {e}")
```

2. The completion summary that gets sent (`bilingual_etl/scripts/main_etl.py`)

The summary is built from the actual processing results:

```
def run(force: bool = False, limit: int | None = None, workers: int = 1):
    # ... (process all categories and companies) ...

    # Build the summary with concrete numbers
    completion_summary = {
        "phase": "complete",
```

```

    "status": "complete",
    "categories": {
        "processed": cat_processed,    # e.g., 127
        "skipped": cat_skipped,        # e.g., 340 (hash unchanged)
        "failed": cat_failed,          # e.g., 2
    },
    "companies": {
        "processed": comp_processed,    # e.g., 89
        "skipped": comp_skipped,        # e.g., 1200
        "failed": comp_failed,          # e.g., 0
    },
    "finished_at": int(time.time()),    # Unix timestamp
}

# Store progress in app_config (for the Admin UI's ETL progress page)
_update_etl_progress(conn, completion_summary)

# Send webhook notification to the configured URL (if any)
_send_etl_webhook(conn, completion_summary)

```

The same summary object serves two purposes: it updates the ETL progress indicator in the Admin UI (via `_update_etl_progress`) and becomes the webhook payload (via `_send_etl_webhook`). This ensures the Admin UI and the webhook recipient always see the same data.

3. Configuring the webhook URL via the Admin UI

The webhook URL is stored in the `app_config` table as a regular configuration entry. The Admin UI uses the generic config CRUD endpoint to read and write it:

```

# In search_api/routers/admin_config.py – this is the generic endpoint

@router.put("/{key}", response_model=ConfigEntry)
def set_config(key: str, body: ConfigValue):
    # When the admin sets key="etl_webhook" with
    # value={"url": "https://hooks.slack.com/services/..."}
    # it writes to app_config, and the next ETL run will
    # read that URL from _send_etl_webhook().
    conn = get_connection()
    try:
        with conn.cursor() as cur:
            cur.execute(
                """
                INSERT INTO app_config (key, value, updated_at)
                VALUES (%s, %s, now())
                ON CONFLICT (key) DO UPDATE

```

```

        SET value = EXCLUDED.value, updated_at = now()
        RETURNING key, value
        """,
        (key, json.dumps(body.value)),
    )
    row = cur.fetchone()
    conn.commit()
finally:
    conn.close()
return ConfigEntry(key=row[0], value=row[1])

```

No dedicated webhook endpoint is needed — the generic `PUT /admin/config/etl_webhook` with body `{"value": {"url": "https://example.com/hook"}}` does the job. The ETL reads it at completion time using a direct SQL query.

Data flow

```

Admin UI
|
| PUT /admin/config/etl_webhook {"value": {"url": "https://..."}}
v
app_config table
key: "etl_webhook"
value: {"url": "https://hooks.slack.com/services/..."}

(later, ETL pipeline runs...)

main_etl.py
|
| 1. Process all categories and companies
| 2. Build completion_summary
| 3. _update_etl_progress(conn, summary) --> writes to app_config
| 4. _send_etl_webhook(conn, summary)
|
| |
| | SELECT value->>'url' FROM app_config WHERE key = 'etl_webhook'
| | POST https://hooks.slack.com/services/... {summary JSON}
v
External system receives notification

```

How to explain this in an interview

"We implemented a webhook notification system for ETL pipeline completion. Instead of requiring external systems to poll for status, the ETL sends an HTTP POST with a JSON summary — including processed, skipped, and failed counts for

both categories and companies — to a configurable URL stored in the database. The URL is managed through the Admin UI's generic configuration CRUD endpoint, so operators can point notifications at Slack, a monitoring service, or any HTTP endpoint without code changes. The webhook uses fire-and-forget semantics with a 10-second timeout: failures are logged as warnings but never block or roll back the pipeline, because the ETL data is already safely committed to the database. This is the opposite of polling — the server notifies the client exactly once at exactly the right time, eliminating wasted requests."